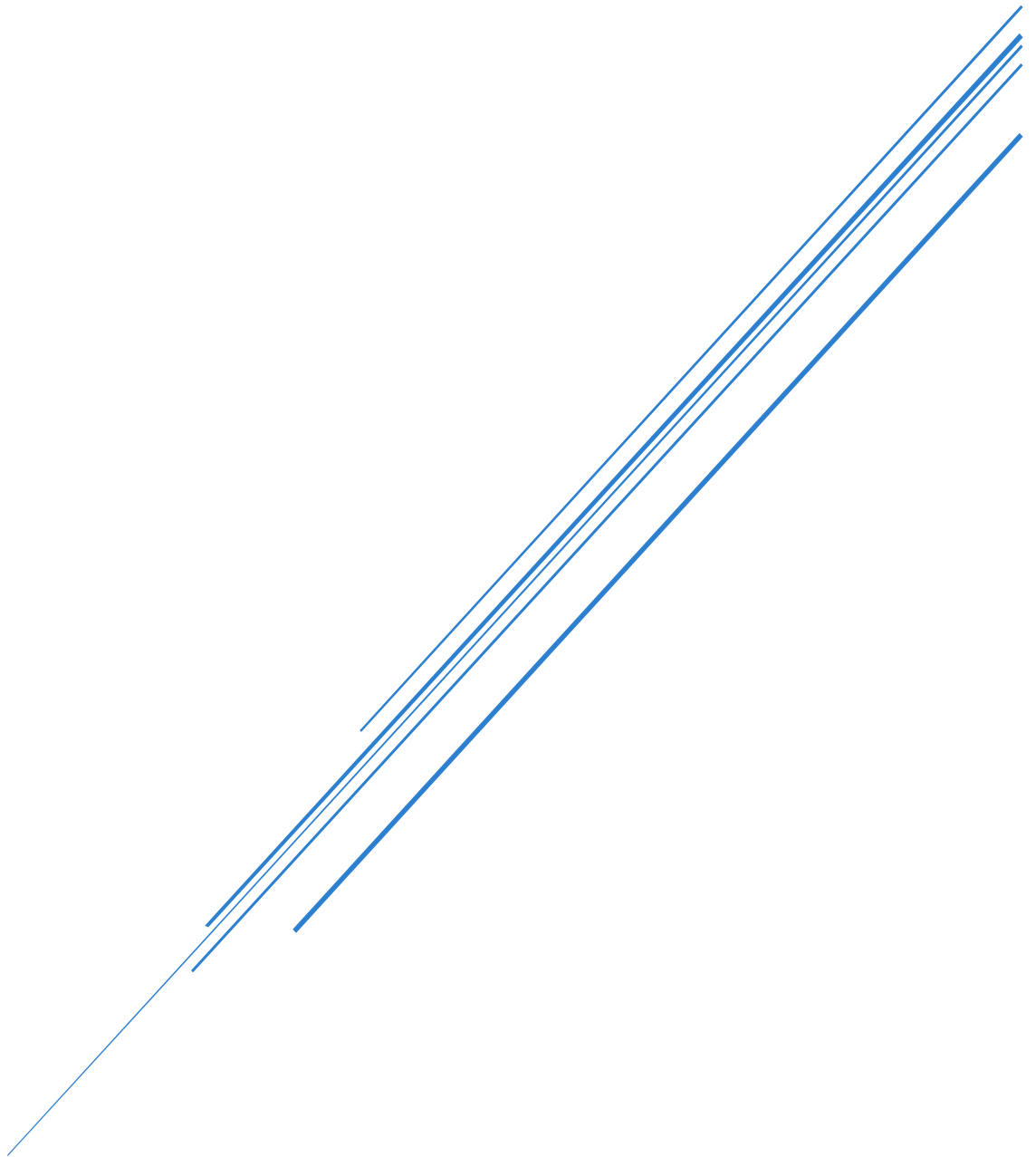


TORQ

Requirement Specification for a web app called TORQ.



Contents

Application Purpose	2
Requirements	8
Functional Requirements.....	8
Non-functional Requirements.....	9
Target Audience	3
User Profiles	3
Timeline	5

Application Purpose

TORQ is a web-based application design to support the motorcycle community, new and old riders, in bike maintenance, safety improvements, and to engage the rider community within Glasgow. The application aims to allow users to:

- Track motorcycle maintenance tasks with dates. Users will input their current bike knowledge, and an algorithm will determine an estimate for next maintenance checks. For example, oil in 3 months, tyre pressure weekly etc.
- Receive reminders on the dashboard for upcoming servicing.
- Users can **optionally** input their MOT, Tax and Insurance renewal dates, and it will display on their user profile.
- A tick able pre-ride safety checklist (useful for new riders) detailing all the quick checks they should do before they ride. This will reset on reload or logout.
- Display local events with days, times, info, and pictures. (these can be filled with placeholder info. Must include event Name, Date, start and end time, description, and IMG for posters)
- Display hot spots, places where bikes regularly accumulate, nice views, ride routes etc.
- Display guides on common bike issues such as how to change oil, how to check brakes, etc.

The system will be data-driven, using user input to dynamically generate personalised outputs.

The goal of TORQ is to improve rider safety and awareness of their bike. Especially for new and younger riders who are just joining the world of motorcycling and may not quite understand their bike. It will also be useful to the older generation of riders who find themselves picking up bad habits or simply forgetting the basics of bike maintenance. It can be a very useful all-in-one tool for people to keep themselves safe and engage within the community via meetups, events, rides.

Target Audience

The application is aimed at a wide range of motorcycle riders including:

- New riders (16+) who are inexperienced in maintenance.
- Non-riders (possibly under 16) who want to know how to obtain a motorcycle license.
- Intermediate riders who require reminders and organisation tools
- Experienced riders who wish to track service history and engage with the local community
- Riders who don't need to track maintenance but want to browse ride outs.

The system will be designed to be accessible to users of varying technical ability through a clean, minimal interface. The web-app will be developed as accessible as possible, adhering to WCAG guidelines.

User Profiles

User Profile 1	
Name	Jamie
Age	19
License Type/Experience	Recently passed CBT and bought 1 st 125cc bike
Needs:	Simple pre ride checklist to ingrain the information until it becomes second nature. Has never owned a bike or car before so doesn't know what the necessary steps are to look after it. Would like to see features that are focused around their safety and making them a better rider.
User Profile 2	
Name	Callum
Age	32
License Type/Experience	A license. Owns multiple bikes. Does own servicing.
Needs:	Needs to track the service history and log maintenance. Is always looking for a new meet up spot, goes to as many events as possible, wants to see them all in the place instead of searching for them.
User Profile 3	
Name	Sarah

Age	27
License Type/Experience	A license. Owned her bike for a few years. Not consistent with upkeep.
Needs:	Ease of use reminders. Would like a calendar view if possible. A simple interactive interface
User Profile 4	
Name	Eddie
Age	65
License Type/Experience	A license. Hasn't owned a bike in 15 years but wants back on the road. Looked after own bikes basic maintenance.
Needs:	Not tech savvy so needs simplistic UI. All his old hotspots are quiet and wants new places to explore. Doesn't know anyone with a bike so would explore the community aspect. Has forgotten a lot of basic maintenance so links to guides and a logger would be useful but not essential.

Timeline

Week 1 – 2	• Write up proposal. • Research the needs within the bike community (done within a whatsapp group chat of rider friends) • Research how to implement certain features. • Create initial wireframes. • Requirement Spec
Week 3 – 4	• Design the database and how it all links together. • Set up the backend (Node.js and express) • Create the database on MySQL and connect it to the backend just to make sure it is working. • Potentially create the info and events pages as they only generate dynamically and do not update in real time like the user dashboard.
Week 5 – 6	• Finalise the user authentication system. • Ensure it is secure. • Begin creating the framing for the frontend with React. • Make it look pretty. • Create the calculation logic for the maintenance tracking system. I.e next service due Feb 13th.
Week 7 – 8	• Create the user dashboard and populate dynamically. • Implement features.
Week 9	Continue development.
Week 10	• Testing and debugging. • Find peers in the community to test. • Create a survey for users to answer. • Collect and collate findings. • Fix any bugs after testing and retest. • Begin evaluation
Week 11	Finish evaluation.

Design Implications

The design of TORQ must consider the needs of a wide and varied audience, ranged ages 16-70.

- The interface will follow a **minimalist, modern design approach** to ensure usability for non-technical users
- A **default dark mode theme** will be implemented to reduce eye strain and provide a modern appearance
- The dashboard will use a **card-based layout** to present key information clearly
- Buttons and interactive elements will be **large, bold and clearly labelled** to improve accessibility
- Navigation will be simple, familiar, and responsive to different viewports.
- The application will be fully **responsive**, ensuring usability on mobile, tablet, and desktop. It will be developed with a mobile first approach.
- High contrast colours will be used to improve readability
- The system will prioritise **ease of use over complexity**, particularly for new riders

Colour Palette and Branding

The brand mark is a white helmet icon paired with the wordmark TORQ. The dark mode theme uses a near-black dark blue rather than pure black or grey, to feel more branded and less generic:

- Page background: near-black dark blue (#0A0E1A)
- Card surface: dark navy (#131B2E), sitting one step lighter than the page background so cards are visible without borders or shadows
- Primary text: near-white (#F5F7FA); secondary and supporting text: muted blue-grey (#8B95AC)
- Accent colour: orange (#FF7A33), used **only** where it carries meaning – the active navigation tab, urgent maintenance alerts, click and hover states, progress indicators, and card border accents. It is never used decoratively, so it keeps its visual weight as a signal for "this needs attention" or "this is active".
- A secondary colour (teal, #5DCAA5) is used only on the calendar to distinguish bike meets from maintenance dates (FR14), keeping orange reserved for maintenance and interactive elements

Dashboard Layout

The home dashboard is split into two groups of cards, reflecting the split between personal bike upkeep and community content:

- **Your bike** – maintenance notification, service and MOT, and pre-ride checklist. Each is a tappable card that opens its own dedicated area of the app, rather than showing full detail on the dashboard itself.
- **Community** – upcoming events and meetups, local rides, and guides.

The bike profile and account settings do not appear on the home dashboard; they are accessed from the user's Profile instead, keeping the home screen focused on things the user needs to act on or check regularly.

On mobile, both groups are laid out as a single stacked column, in keeping with the mobile-first approach. On tablet and desktop, the extra width is used for a persistent right-hand sidebar rather than simply widening the cards. The sidebar contains:

- A bike snapshot card showing make, model, year, mileage and last service date at a glance
- A mini calendar (supporting FR13/FR14) with colour-coded highlighted dates – orange for maintenance, teal for bike meets – and a small legend beneath it

The main content area (Your bike and Community cards) remains a single column on mobile and reflows into the space beside the sidebar on desktop, so the same information architecture holds across breakpoints; only the sidebar is desktop-only.

Development Lifecycle

The project will follow an Agile development approach, allowing for iterative development and improvement. I believe this will benefit me and my learning the best, as it allows me to be ambitious but also keep within my abilities.

Development will be broken into stages (planning, development, testing, evaluation). Features can be created modularly and implemented in increments. Testing can occur throughout development, allowing me to focus on tasks and at the end before submission, fix any bugs fine tune the design etc. Feedback will be gathered throughout the project and importantly, at the end, and will be used to refine the overall design and functionality of the app.

Requirements

Functional Requirements

FR1	Allow users to log in securely
FR2	Allow users to create and manage a bike profile
FR3	Store maintenance records
FR4	Calculate future maintenance from user input
FR5	Display maintenance reminders on dashboard
FR6	Provide interactive pre-ride checklist
FR7	Reset checklist on reload and logout
FR8	Display local events and hotspots dynamically
FR9	Display guides on maintenance, possibly hyperlinking to other sites
FR10	Display important vehicle dates such as MOT, insurance
FR11	Store and retrieve user specific user data from a database
FR12	Personalised user dashboard
FR13	Have a calendar on the dashboard with dates highlighted. When clicked it will populate a card underneath with the information. (Such as MOT due, oil change due etc)
FR14	Calendar highlights will be colour coded (Bike meet, maintenance)
FR15	Display today's weather conditions on the dashboard, to support pre-ride safety decisions
FR16	Provide search and filter within Guides, Events and Hotspots, so users can find content directly rather than only browsing
FR17	Show a clear empty state (e.g. "Add your bike") on the dashboard and bike snapshot when a new user has not yet created a bike profile
FR18	Display a "last synced" timestamp on cached bike specification data, so users know how current the information is
FR19	Use a consistent toast or inline confirmation pattern for actions such as logging maintenance or resetting the checklist
FR20	Show clear loading and error states for any dynamically fetched content (events, hotspots, bike specification data)
FR21	Allow new users to register an account
FR22	Allow users to securely log out, clearing session data
FR23	Allow users to view and edit a personal profile (display name, avatar), separate from their bike profile(s)
FR24	Allow users to add and manage multiple bikes under one account, with one bike set as the default shown on the dashboard
FR25	Send automated email reminders to users ahead of MOT, tax and insurance renewal dates
FR26	Show a rider-friendliness indicator (Good, caution, poor) on the dashboard, calculated from live weather conditions

Non-functional Requirements

NFR1	Be responsive across mobile, tablet, desktop devices
NFR2	Load within 2 seconds under normal conditions
NFR3	Consistent interface, dark mode for an easy look on the eyes
NFR4	Securely store passwords using hashing
NFR5	User-friendly and easy to navigate
NFR6	Follow WCAG accessibility standards
NFR7	Be accessible to a variety of users
NFR8	Maintain data integrity, privacy and consistency
NFR9	Smooth interactions and transitions
NFR10	Degrade gracefully when dynamically fetched content (weather, events, bike specification data) is slow or unavailable, showing loading and error states rather than a blank or broken screen
NFR11	Cache external API responses (motorcycle specifications, weather) where appropriate, to reduce repeat calls and stay within free-tier usage limits

System Requirements

Hardware Requirements

- Computer, laptop or mobile
- Internet connection
- Peripherals

Foundations (language, DBMS)

The application will be developed using React, Node.js With Express, MySQL and Tailwind for styling. All of these work together well, are scalable and flexible, and relevant to the current industry.

Development Requirements

- Node.js (Backend)
- Express.js (API handler)
- React (Frontend)
- MySQL (Database)
- XAMPP (Local server)
- Visual Studio Code (IDE)
- Git for version control
- Trello (Project management)
- Figma and Figjam (wireframing)
- Nodemailer with an SMTP provider (email reminders, FR21)
- Open-Meteo (weather data, FR15/FR26)

API Ninjas Motorcycle API

What It Does

API Ninjas provides a free, publicly accessible Motorcycles API containing specification data on tens of thousands of motorcycle models from hundreds of manufacturers. It exposes three relevant endpoints:

- `/v1/motorcyclemakes` – returns a list of all available manufacturers
- `/v1/motorcyclemodels?make=X` – returns all models for a given make
- `/v1/motorcycles?make=X&model=Y&year=Z` – returns full technical specifications for a specific make, model and year, including engine, power, torque, displacement and dimensions

The API is used under its free tier, which permits personal, non-commercial, development and evaluation use – appropriate for this project’s scope.

Role in the App

The API powers the “My Bike” profile creation flow (FR2). When a user adds a bike, they select their bike’s make, model and year from cascading dropdown menus rather than typing free text, ensuring consistent, accurate data. The retrieved specification data is also used to populate an “All About My Bike” summary card on the user dashboard (FR12), giving each user a personalised, data-driven overview of their motorcycle alongside their maintenance information.

Implementation

- On the bike profile form, the Make dropdown is populated by calling `/v1/motorcyclemakes`.
- Selecting a make populates the Model dropdown via `/v1/motorcyclemodels?make=X`.
- Year is selected from a filtered range or from the results of the specifications endpoint.
- On submission, the backend calls `/v1/motorcycles` once with the chosen make, model and year, and stores the returned JSON specification data in the bikes table in MySQL, linked to the user’s account.
- Caching the response at creation time, rather than calling the API on every dashboard load, keeps the dashboard within the 2-second load target (NFR2) and keeps the app functional even if the third-party API is temporarily unavailable.
- The API key is stored as a server-side environment variable and never exposed to the frontend, since all calls to the Motorcycles API are proxied through the Express backend.

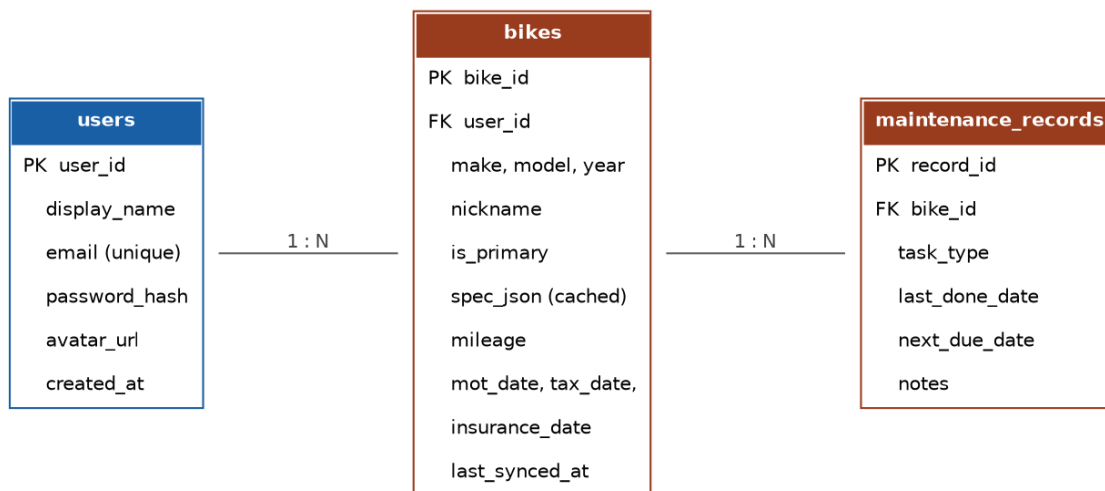
The API does not reliably provide bike images. Until an image source is confirmed, the bike profile will display a generic icon based on bike type (e.g. sport, cruiser, adventure, naked) as a placeholder, since a photo is a desirable but non-essential feature.

Database Design

The database will be implemented in MySQL, matching the Foundations section. The design below covers the core entities needed for FR2–FR5, FR10–FR14 and FR24, and reflects a change from earlier planning: since User Profile 2 (Callum) owns multiple bikes and does his own servicing, a user is modelled as having many bikes rather than one, with a single bike flagged as the default shown on the dashboard (FR24).

Entity Relationship Diagram

The three core relational tables and their keys are shown below. Independent reference tables are described separately, as they do not hold foreign keys to a user.



Independent Reference Tables

These tables hold data that is not owned by an individual user, so they carry no foreign key back to the users table:

- **checklist_items** (item_id PK, label, sort_order) – the static pre-ride checklist items (FR6). Completion state is held in front-end session state only and is not written back to this table, since FR7 requires the checklist to reset on reload and logout.
- **events** (event_id PK, name, description, event_date, start_time, end_time, image_url, latitude, longitude) – supports FR8, populated initially with placeholder data as noted in the Application Purpose section.
- **hotspots** (hotspot_id PK, name, description, category, latitude, longitude) – supports FR8, and pairs with the Geolocation API research noted in Constraints.
- **weather_cache** (cache_id PK, latitude, longitude, fetched_at, expires_at, raw_json) – supports NFR11, discussed further in the Weather API section below.

Design Notes

- bikes.spec_json stores the cached API Ninjas response (see API Ninjas Motorcycle API section), so a bike's specification data survives even if the third-party API is temporarily unavailable.

- maintenance_records is one-to-many from bikes rather than users, so multi-bike accounts (FR24) keep separate service histories per bike, which matters for Callum's persona.
- mot_date, tax_date and insurance_date are nullable, since these fields are optional per the Application Purpose section.
- Passwords are never stored in plain text; users.password_hash holds a bcrypt hash only, per NFR4.

Weather API

What It Does

Open-Meteo is used to provide the dashboard weather data. It is free for non-commercial use, requires no API key or account, and returns current conditions plus an hourly forecast as plain JSON over a simple HTTP GET request. Relevant fields are temperature, precipitation probability, wind speed and gusts, and a weather code describing conditions such as fog, rain or thunderstorms. Not needing an API key removes a credential to manage and secure, which fits well alongside the API Ninjas integration.

Role in the App

The dashboard displays today's conditions for the user's area, defaulting to Glasgow (FR15). Alongside the raw conditions, a simple rider-friendliness indicator is calculated and shown as Good, caution, or poor (FR26), directly supporting the pre-ride checklist and the goal of improving rider safety.

Ride-Suitability Algorithm

A simple rule-based approach is used rather than a machine-learning model, since the goal is an explainable, easily tested indicator rather than a predictive one. Each factor is scored independently, and the overall result takes the single worst (most restrictive) factor, rather than an average, so that one dangerous condition cannot be masked by otherwise good weather:

- **Temperature** – good above 10°C; caution 5–10°C (cold, reduced grip and dexterity); poor below 5°C (risk of ice)
- **Precipitation probability** – good below 30%; caution 30–60%; poor above 60%
- **Wind speed** – good below 25 km/h; caution 25–40 km/h; poor above 40 km/h, or if gusts are significantly higher than sustained speed
- **Weather code** – thunderstorm, snow or ice conditions are automatically scored as poor regardless of the other factors

This is a first-pass rule set based on general riding advice rather than tested guidance, and the thresholds should be reviewed and adjusted during testing (Week 10) with feedback from the riders being surveyed.

Implementation

- The Express backend calls Open-Meteo's forecast endpoint with the user's stored latitude and longitude (defaulting to Glasgow if not set), requesting current conditions plus the next few hours.
- The response is cached in the `weather_cache` table, keyed by rounded latitude and longitude, with a short expiry (for example fifteen to thirty minutes), so multiple users in the same area do not each trigger a fresh external call, and the dashboard keeps loading within NFR2 even if Open-Meteo is briefly slow.
- The ride-suitability score is calculated on the backend from the cached data, not in the browser, so the scoring rules are consistent and easy to update in one place.
- If Open-Meteo is unavailable, the dashboard shows the last cached reading with its timestamp, or a clear error state if no cached reading exists, per FR20 and NFR10.

Project Plan

The project will be managed on Trello and developed over an 11 week period.

Week 1 – 2	• Write up proposal. • Research the needs within the bike community (done within a whatsapp group chat of rider friends) • Research how to implement certain features. • Create initial wireframes. • Requirement Spec
Week 3 – 4	• Design the database and how it all links together. • Set up the backend (Node.js and express) • Create the database on MySQL and connect it to the backend just to make sure it is working. • Potentially create the info and events pages as they only generate dynamically and do not update in real time like the user dashboard.
Week 5 – 6	• Finalise the user authentication system. • Ensure it is secure. • Begin creating the framing for the frontend with React. • Make it look pretty. • Create the calculation logic for the maintenance tracking system. I.e next service due Feb 13th.
Week 7 – 8	• Create the user dashboard and populate dynamically. • Implement features.
Week 9	Continue development.
Week 10	• Testing and debugging. • Find peers in the community to test. • Create a survey for users to answer. • Collect and collate findings. • Fix any bugs after testing and retest. • Begin evaluation
Week 11	Finish evaluation.

Navigation Structure

The application will include:

- Home
- Dashboard / My account / Login (when not logged in)
- Guides
- Events
- Hotspots
- Pre-ride checklist (links to my account)

Navigation will be designed different depending on viewport. Mobile will feature a drop down menu via hamburger, whereas tablet and desktop will have a horizontal list in the top right corner.

Constraints

- Time limitations – I only have until May 29th to develop the site.
- User interactivity will be somewhat limited, being unable to post like a social media feed.
- Data is only as accurate as the users input.
- Application is likely to be focused within the one region, however in future could be expanded.
- Need to learn more about Geolocation APIs as this will help display locations for meet ups.
- Heavily data driven, although I have worked with Node, MySQL and express before, it was tough. I risk spending more time debugging things I am unfamiliar with.
- I have very limited experience with React as a frontend framework, so will have to learn as I go.
- I don't have the sharpest skills when it comes to design, creating logos or graphics.
- User data must be protected, and full company level security is out of project scope

What I will learn

The development of TORQ will require a lot of learning, improving and honing in skills. These include:

- React framework and its component based architecture.